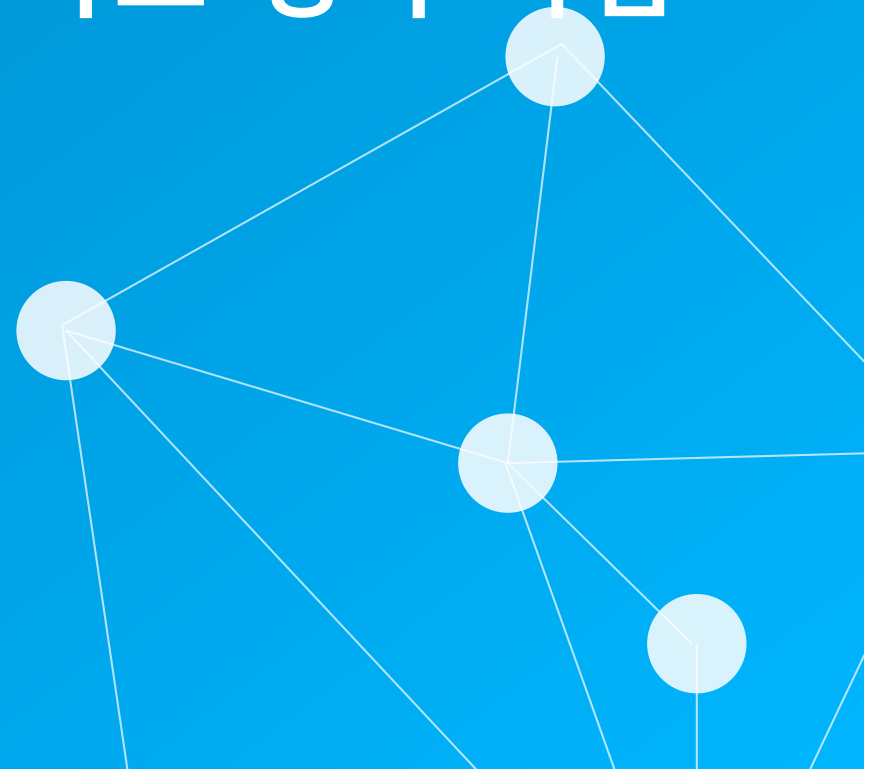


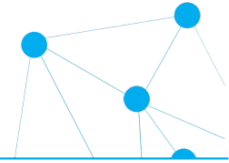
04

CHAPTER

축소 정복 기법

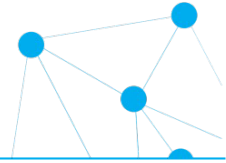


학습 내용



- 4.1 삽입 정렬(insertion sort)
- 4.2 위상 정렬
- 4.3 이진 탐색
- 4.4 거듭제곱 문제
- 4.5 선택 문제: k번째 작은 수 찾기
- 4.6 축소 정복 기법의 추가적인 예

축소 정복 기법(decrease-and-conquer)



- 주어진 문제와 더 작은 문제간의 관계를 이용하는 전략
 - 예: $n! = n * (n-1)!$
 - 분할 정복 기법의 일종
- 적용 방법
 - 하향식(top-down): 순환구조
 - 상향식(bottom-up): 반복구조

상향식

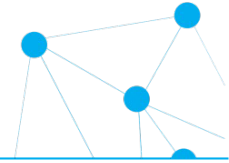
알고리즘 4.1 팩토리얼(하향식 축소 정복 기법)

```
01 def factorial_recur(n) :  
02     if n == 1 :  
03         return 1  
04     else :  
05         return (n * factorial_recur(n - 1))
```

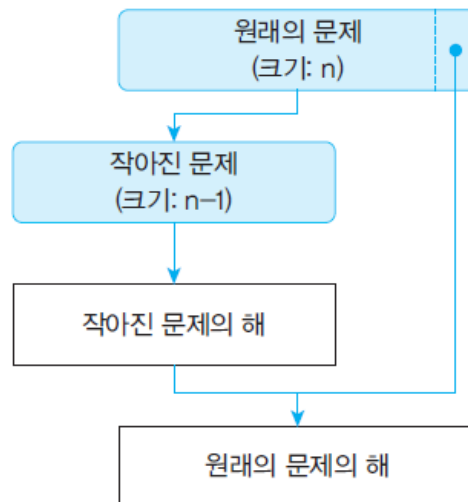
알고리즘 4.2 팩토리얼(상향식 축소 정복 기법)

```
01 def factorial_iter(n) :  
02     result = 1  
03     for k in range(1,n+1) :  
04         result = result * k  
05     return result
```

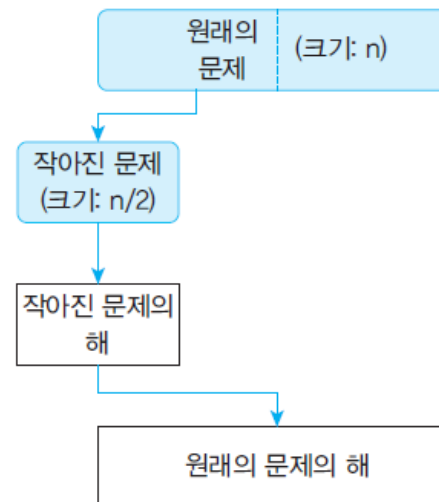
문제 축소 형태



- 고정 크기 축소(decrease by a constant):
 - 예: 팩토리얼
- 고정 비율 축소(decrease by a constant factor):
 - 예: 이진 탐색
- 가변 크기 축소(variable size decrease):
 - 예: 유클리드 알고리즘

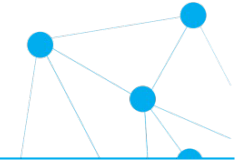


고정 크기(1) 축소 정복 기법
(decrease by a constant)

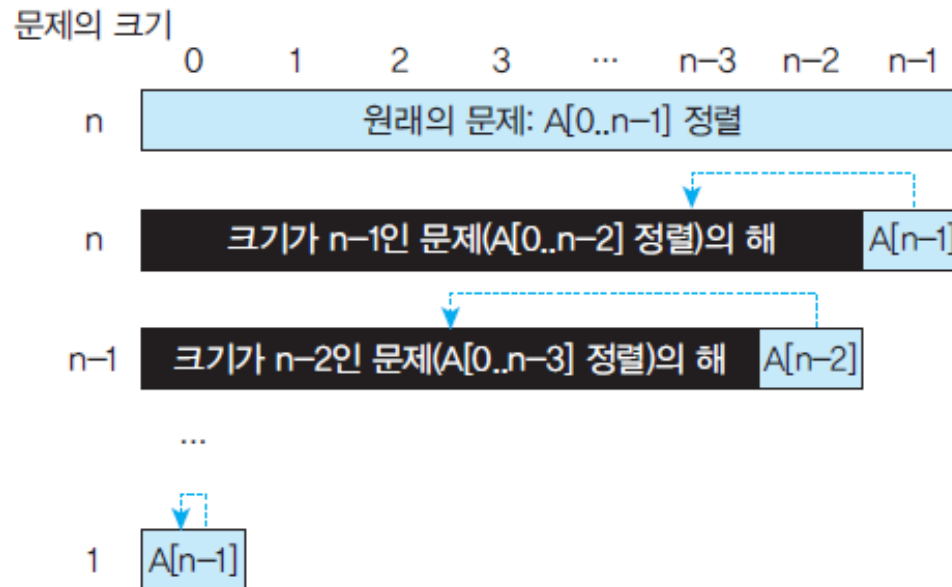


고정 비율(1/2) 축소 정복 기법
(decrease by a constant factor)

4.1 삽입 정렬(insertion sort)



- 정렬을 위한 축소 정복 전략
 - $A[0..n-1]$ 의 정렬 $\rightarrow A[0..n-2]$ 의 정렬 + $A[n-1]$ 을 끼워 넣기



원래의 문제: $A[0..n-1]$ 을 정렬

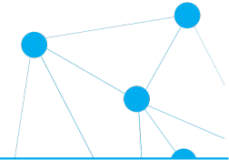
원래의 문제는 $A[0..n-2]$ 이 정렬되면
 $A[n-1]$ 을 적절한 위치를 끼워 넣는 문제와 같음

$A[0..n-2]$ 정렬 문제는 $A[0..n-3]$ 이 정렬되면
 $A[n-2]$ 을 적절한 위치를 끼워 넣는 문제와 같음

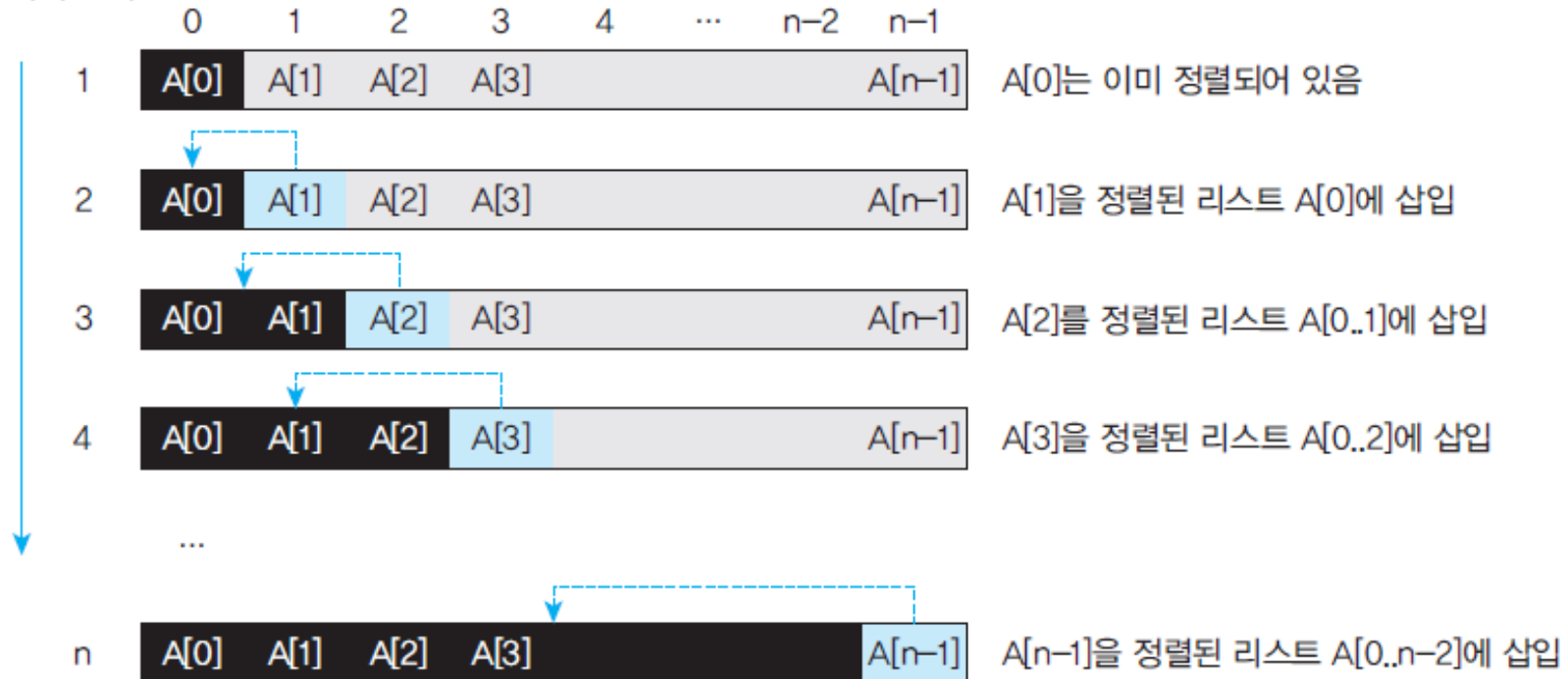
$A[0]$ 정렬 문제는 항목이 하나이므로
 이미 해결된 문제임

[그림 4.3] 정렬을 위한 축소 정복 전략

상향식 축소 정복 전략



문제의 크기



[그림 4.4] 정렬을 위한 축소 정복 전략(상향식)

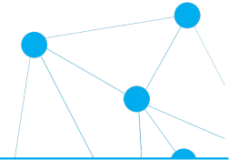
효율적인 끼워 넣기를 위한 삽입 위치 찾기

- 뒤에서 앞으로 넣을 위치를 찾아나감
 - 항목을 미리 뒤로 옮길 수 있음



[그림 4.7] 정렬된 부분 리스트에 항목을 끼워 넣는 과정

알고리즘



알고리즘 4.3 삽입 정렬

```
01 def insertion_sort(A) :  
02     n = len(A)  
03     for i in range(1, n) :  
04         key = A[i]  
05         j = i-1  
06         while j>=0 and A[j] > key :  
07             A[j + 1] = A[j]  
08             j = j - 1  
09         A[j + 1] = key  
10     printStep(A, i)
```

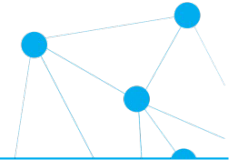
알고리즘 테스트 삽입 정렬

```
data = [ 5, 3, 8, 4, 9, 1, 6, 2, 7 ]  
print("Original :", data)  
insertion_sort(data)  
print("Insertion :", data)
```

```
C:\WINDOWS\system32\cmd.exe  
Original : [5, 3, 8, 4, 9, 1, 6, 2, 7]  
Step 1 = [3, 5, 8, 4, 9, 1, 6, 2, 7]  
Step 2 = [3, 5, 8, 4, 9, 1, 6, 2, 7]  
Step 3 = [3, 4, 5, 8, 9, 1, 6, 2, 7]  
Step 4 = [3, 4, 5, 8, 9, 1, 6, 2, 7]  
Step 5 = [1, 3, 4, 5, 8, 9, 6, 2, 7]  
Step 6 = [1, 3, 4, 5, 6, 8, 9, 2, 7]  
Step 7 = [1, 2, 3, 4, 5, 6, 8, 9, 7]  
Step 8 = [1, 2, 3, 4, 5, 6, 7, 8, 9]  
Insertion : [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

- 입력의 크기?
- 기본 연산?

복잡도 분석



- 입력 구성에 따른 차이가 있음

- 최선의 경우: $O(n)$

$$T_{best}(n) = (n-1) \cdot 1 = n-1$$

- 정렬된 리스트?

- 최악의 경우: $O(n^2)$

$$T_{worst}(n) = \sum_{i=1}^{n-1} \sum_{j=0}^{i-1} 1$$

- 역으로 정렬된 리스트?

$$= \sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} O(n^2)$$

- 평균적인 경우 : $O(n^2)$

$$T_{avg}(n) \approx \frac{n^2}{4} \approx \frac{1}{2} T_{worst}(n) \in O(n^2)$$

- 최악의 경우의 절반 정도

- 특징

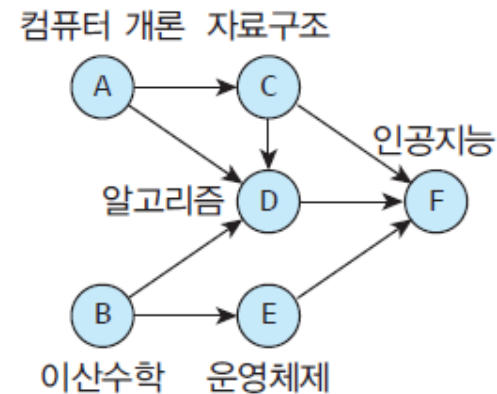
- 특히 많은 항목(레코드)을 이동이 필요
- 레코드가 이미 정렬되어 있는 경우 효율적 → 셸(Shell) 정렬

4.2 위상 정렬

방향 그래프 $G = (V, E)$ 가 주어졌다. G 에 존재하는 각 정점들의 선행 순서를 위배하지 않으면서 모든 정점들을 순서대로 나열하라.

과목 번호	과목명	선수과목
A	컴퓨터 개론	없음
B	이산수학	없음
C	자료구조	A
D	알고리즘	A, B, C
E	운영체제	B
F	인공지능	C, D, E

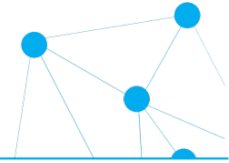
교과목의 선후수 관계표



방향 그래프로 표시한 선후수 관계

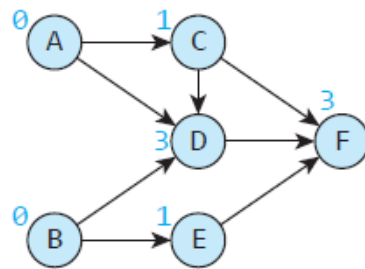
- $\langle A, B, C, D, E, F \rangle$, $\langle B, A, C, D, E, F \rangle$, $\langle A, C, B, E, D, F \rangle$: 가능한 수강 순서
- $\langle C, A, B, D, E, F \rangle$: 가능하지 않은 수강 순서

축소 정복 기법의 알고리즘

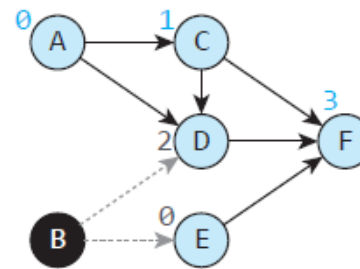


- 알고리즘

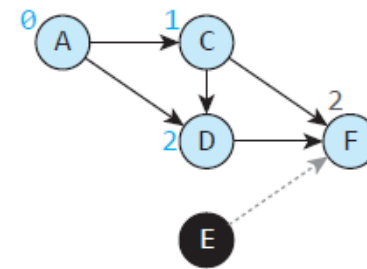
- 진입 차수가 0인 임의의 정점 선택
- 선택된 정점을 삭제, 이 정점에서 진출하는 모든 간선들도 삭제, 차수 갱신, 이 과정을 반복 → 문제의 크기가 1 줄어듦



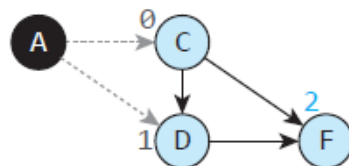
(a) 초기 상태



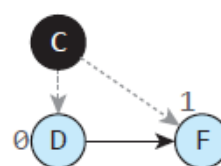
(b) B 삭제



(c) E 삭제



(d) A 삭제



(e) C 삭제



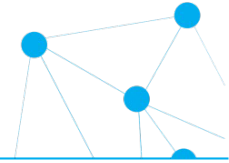
(f) D 삭제



(g) F 삭제.
탐색 종료

[그림 4.10] 위상 정렬 과정의 예: B-E-A-C-D-F

알고리즘



알고리즘 4.4 위상 정렬(축소 정복 기법)

```
01 def topological_sort(graph) :
02     inDeg = {}
03     for v in graph :
04         inDeg[v] = 0
05     for v in graph :
06         for u in graph[v]:
07             inDeg[u] += 1
08
09     vlist = []
10     for v in graph :
11         if inDeg[v]==0 :
12             vlist.append(v)
13
14     while vlist :
15         v = vlist.pop()
16         print(v, end=' ')
17
18         for u in graph[v] :
19             inDeg[u] -= 1
20             if inDeg[u]==0 :
21                 vlist.append(u)
```

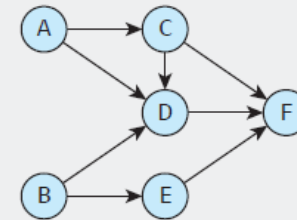
$O(n)$

$O(e)$

$O(n)$

$O(e)$

```
mygraph = { "A" : {"C", "D"},
            "B" : {"D", "E"},
            "C" : {"D", "F"},
            "D" : {"F"},
            "E" : {"F"},
            "F" : {}
          }
```



알고리즘 테스트 위상 정렬

```
mygraph = { "A" : {"C", "D"}, ... }
print('topological_sort: ')
topological_sort(mygraph)
print()
```

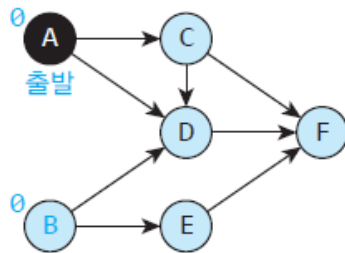
```
C:\WINDOWS\system32\cmd.exe
topological_sort:
B E A C D F
```

$O(n + e)$

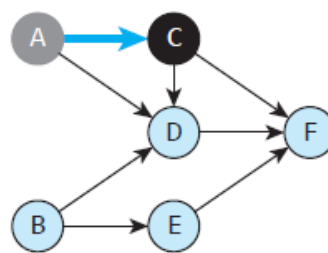
DFS 기반 알고리즘(억지 기법) (틀린 설명)

- 알고리즘

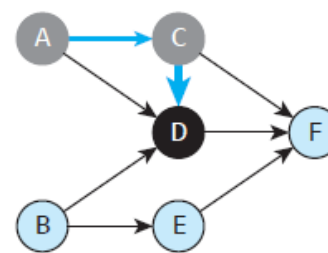
- 진입 차수가 0인 임의의 정점을 시작 정점으로 선택
- 깊이 우선 탐색으로 그래프의 정점들을 방문
- 더 이상 갈 수 있는 정점이 없으면 다른 진입 차수가 0인 정점



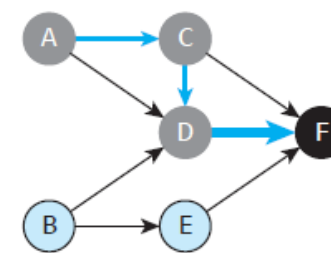
(a) A에서 DFS 출발



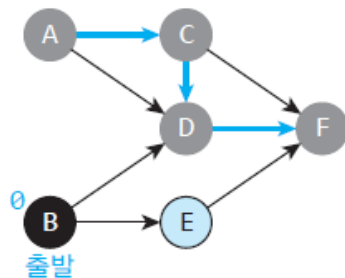
(b) A → C로 이동



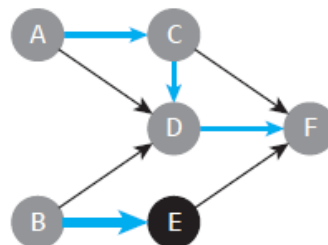
(c) C → D로 이동



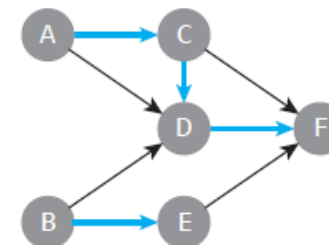
(d) D → F로 이동



(e) B에서 다시 출발



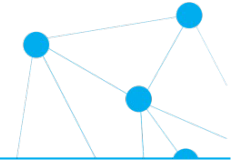
(f) B → E로 이동



(g) 탐색 종료: A → C → D → F → B → E

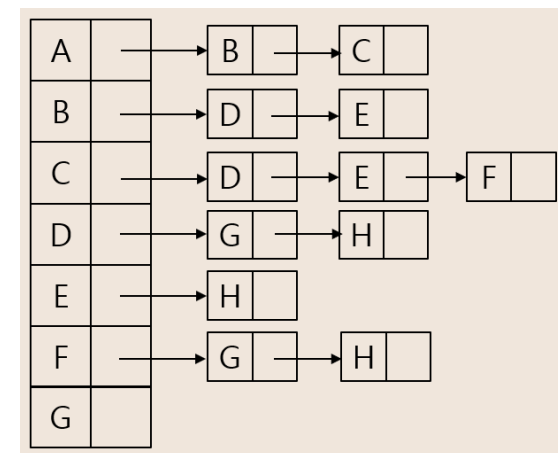
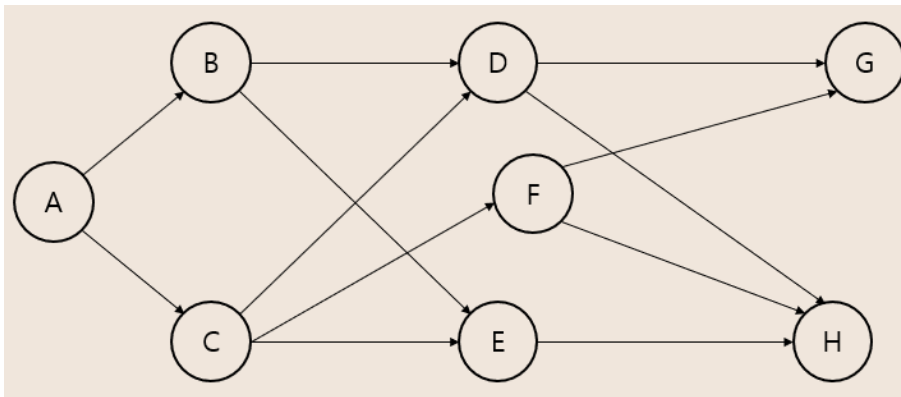
[그림 4.9] DFS 기반 위상 정렬 과정의 예: A-C-D-F-B-E

DFS 기반 알고리즘(억지 기법)

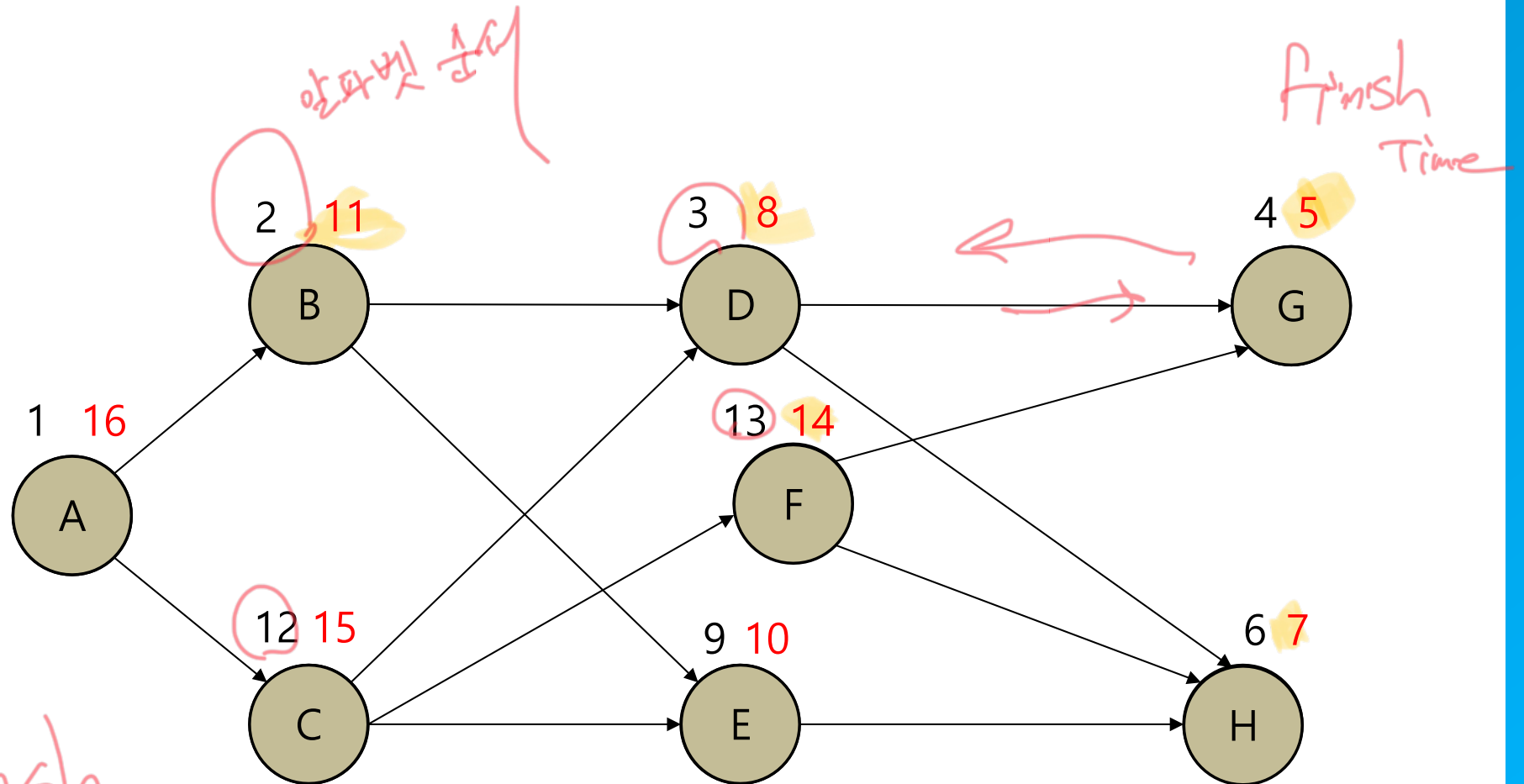
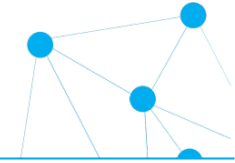


- 알고리즘

- DFS(G) 호출, 각 정점 v 의 종료시각 $f[v]$ 를 계산
- 각 정점이 종료될 때, 그 정점을 list에 추가
- List의 역순을 출력



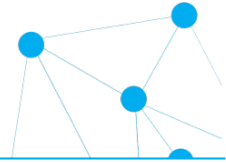
DFS 기반 알고리즘(억지 기법)



List: G H D E B F C A

위상정렬 결과: A C F B E D H G

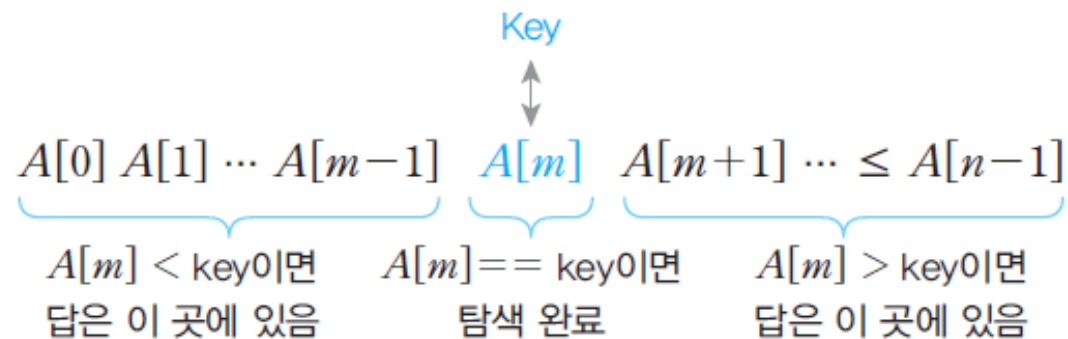
4.3 이진 탐색



문제 4.2 정렬된 리스트에서의 탐색 문제

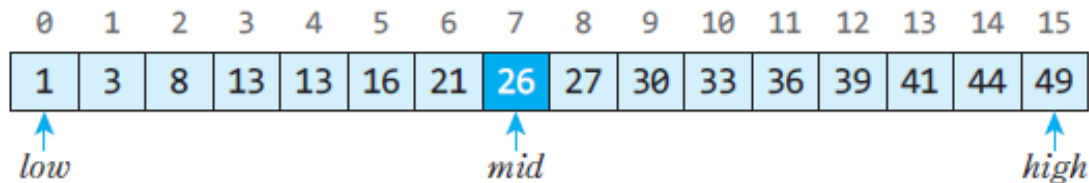
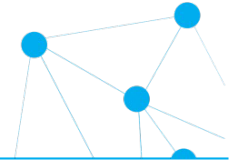
리스트에 n 개의 항목이 들어있다. 이 리스트에서 “탐색키”를 가진 항목을 찾아라. 단, 리스트의 항목들은 정렬되어 있다.

- 축소 정복 탐색 전략
 - 중앙에 있는 항목을 먼저 조사

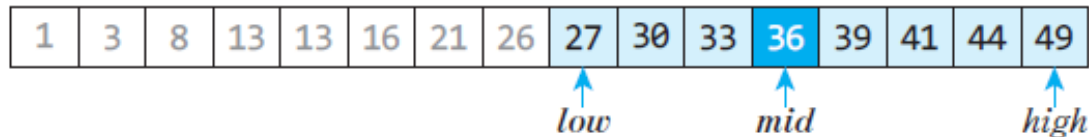


[그림 4.11] 이진 탐색의 문제 축소 과정

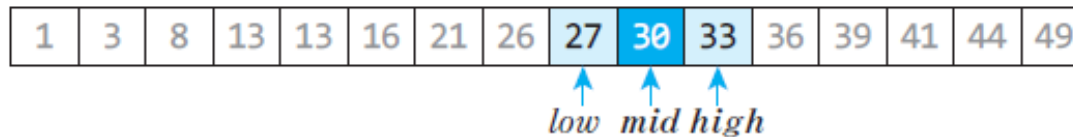
축소 정복 탐색 전략: 이진 탐색



❶ $A[mid] < 33$
 $low \leftarrow mid + 1$



❷ $A[mid] > 33$
 $high \leftarrow mid - 1$



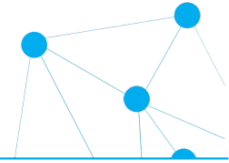
❸ $A[mid] < 33$
 $low \leftarrow mid + 1$



❹ $A[mid] = 33$
 탐색 완료

[그림 4.12] 정렬된 리스트 A에서 33을 이진 탐색으로 찾는 과정

알고리즘



알고리즘 4.6 이진 탐색(반복구조)

```
01 def binary_search_iter(A, key, low, high) :
02     while (low <= high) :
03         mid = (low + high) // 2
04         if key == A[mid]:
05             return mid
06         elif key > A[mid]:
07             low = mid + 1
08         else:
09             high = mid - 1
10     return -1
```

알고리즘 테스트 순환과 반복 구조의 이진탐색 알고리즘

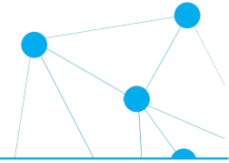
```
listA = [1, 3, 8, 13, 13, 16, 21, 26, 27, 30, 33, 36, 39, 41, 44, 49]
print("입력 리스트 =", listA) # 리스트는 반드시 정렬되어 있어야 한다.
print("33 탐색(순환) -->", binary_search(listA, 33, 0, len(listA)-1) )
print("33 탐색(반복) -->", binary_search_iter(listA, 33, 0, len(listA)-1) )
print("32 탐색(순환) -->", binary_search(listA, 32, 0, len(listA)-1) )
print("32 탐색(반복) -->", binary_search_iter(listA, 32, 0, len(listA)-1) )
```

```
C:\WINDOWS\system32\cmd.exe
입력 리스트 = [1, 3, 8, 13, 13, 16, 21, 26, 27, 30, 33, 36, 39, 41, 44, 49]
33 탐색(순환) --> 10
33 탐색(반복) --> 10
32 탐색(순환) --> -1
32 탐색(반복) --> -1
```

탐색 성공(항목의 인덱스)

탐색 실패(-1)

복잡도 분석



- 입력 구성에 따른 차이가 있음

- $T_{best}(n) \in O(1)$

- 최악: $T_{worst}(n) \in O(\log_2 n)$

- 리스트에 찾는 값이 없는 경우

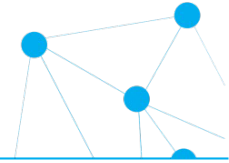
- $k = \log_2 n$

$$\begin{aligned} T_{worst}(n) &= T_{worst}(\lfloor n/2 \rfloor) + 1 \\ &= T_{worst}(1) + k \\ &= 1 + k \end{aligned}$$

- 특징

- 반드시 리스트가 정렬되어 있어야 함
 - 데이터의 삽입이나 삭제가 빈번한 응용에는 적합하지 않음

4.4 거듭제곱 문제



x 의 n -거듭제곱인 x^n 을 계산하라.

- 억지 기법

알고리즘 4.7 거듭제곱(억지 기법)

```
01 def slow_power(x, n) :  
02     result = 1.0  
03     for i in range(n):  
04         result = result * x  
05     return result
```

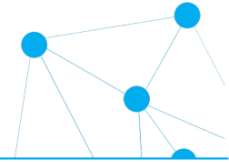
- 축소 정복 아이디어

$$\begin{aligned}x^{10} &= x \cdot x \cdot x \cdot x \cdot x \cdot x \cdot x \cdot x \cdot x \cdot x \\&= (x \cdot x) \cdot (x \cdot x) \cdot (x \cdot x) \cdot (x \cdot x) \cdot (x \cdot x) \\&= (x \cdot x)^5 = (x^2)^5 = y^5 \\x^{10} &= (x^2)^5 = x^2 \cdot (x^2)^4 = x^2 \cdot ((x^2)^2)^2\end{aligned}$$

$$x^n = \begin{cases} 1 & n = 0 \\ (x^2)^{n/2} & n \text{이 짝수} \\ x \cdot (x^2)^{(n-1)/2} & n \text{이 홀수} \end{cases}$$

- 곱셈의 수?

축소 정복 알고리즘



알고리즘 4.8 거듭제곱(축소 정복 기법)

```
01 def power(x, n) :  
02     if n == 0 :  
03         return 1  
04     elif (n % 2) == 0 :  
05         return power(x*x, n//2)  
06     else :  
07         return x * power(x*x, (n-1)//2)
```

C:\WINDOWS\system32\cmd.exe

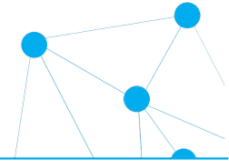
```
억지기법 (2**500) = 3.273390607896142e+150  
축소정복기법 (2**500) = 3.273390607896142e+150  
억지기법 시간... 1.6984951496124268  
축소정복기법 시간... 0.17752671241760254
```

2**500 계산 결과. 동일함

순환으로 구현되었지만

축소 정복 기법이 훨씬 빠름

복잡도 분석



- 순환 호출을 한번 할 때마다 n 이 크기가 절반씩 줄어듦
 - $n = 2^k$
 $2^k \rightarrow 2^{k-1} \rightarrow 2^{k-2} \rightarrow \dots \rightarrow 2^1 \rightarrow 2^0$
 - 복잡도: $O(\log_2 n)$
- 행렬의 거듭제곱 문제
 - 동일한 적용이 가능

```
01 def powerMat(x, n) :  
02     if n == 1 :  
03         return x  
04     elif (n % 2) == 0 :  
05         return powerMat(multMat(x,x), n // 2)  
06     else :  
07         return multMat(x, powerMat(multMat(x,x), (n - 1) // 2))
```

4.5 선택 문제: k 번째 작은 수 찾기

리스트에서 k 번째로 작은 항목을 찾아라. 단, 리스트는 정렬되어 있지 않다.

0	1	2	3	4	5
7	10	4	3	20	15

$k=1$ 가장 작은 수는 3

$k=2$ 두 번째로 작은 수는 4

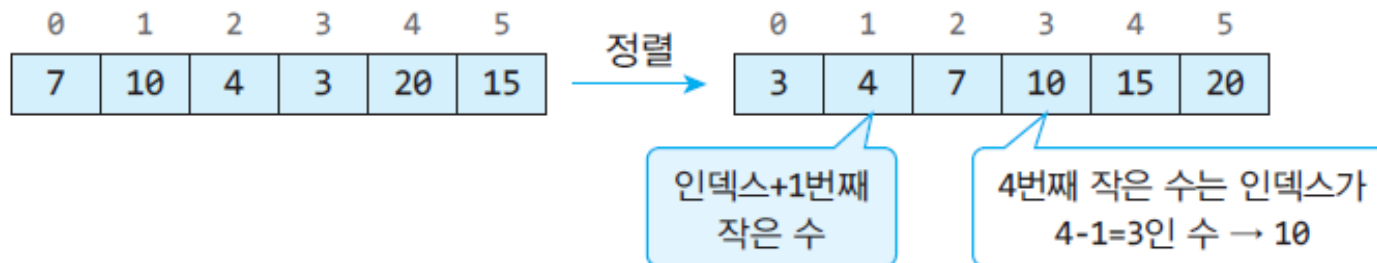
$k=3$ 세 번째로 작은 수는 7

...

$k=6$ 여섯 번째로 작은 수는 20

[그림 4.13] 리스트에서 k 번째로 작은 수를 찾는 선택 문제의 예

- 방법1: 정렬을 이용



- 방법2: 축소 정복 기법 (quick_select)

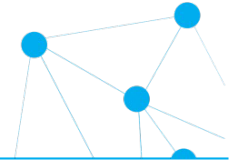
축소 정복을 이용한 k번째 작은 수 찾기

- 리스트의 한 항목을 **피벗(pivot)**으로 선택한다. 보통 리스트의 맨 왼쪽 항목을 선택한다.
- 리스트의 나머지 항목들을 모두 검사하여 피벗보다 작으면 피벗의 왼쪽으로 옮기고 피벗보다 크면 피벗의 오른쪽으로 옮긴다. 결과적으로 피벗을 중심으로 왼쪽은 피벗보다 작은 요소들로 구성되고, 오른쪽은 피벗보다 큰 요소들로 구성된다.

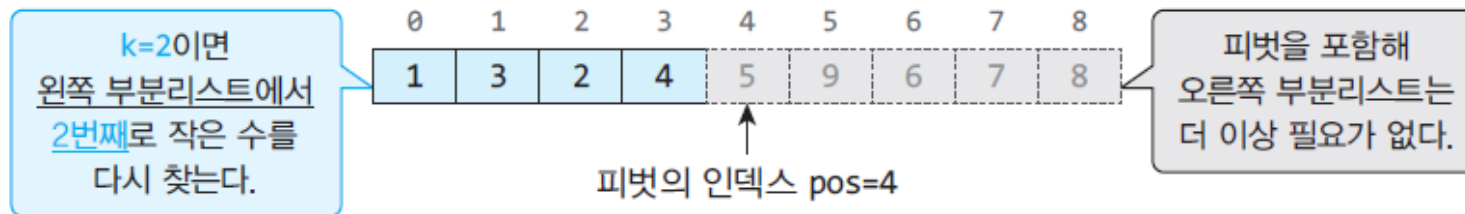


[그림 4.15] 첫 번째 원소로 피벗을 선택하고 피벗을 중심으로 리스트를 분할하는 예

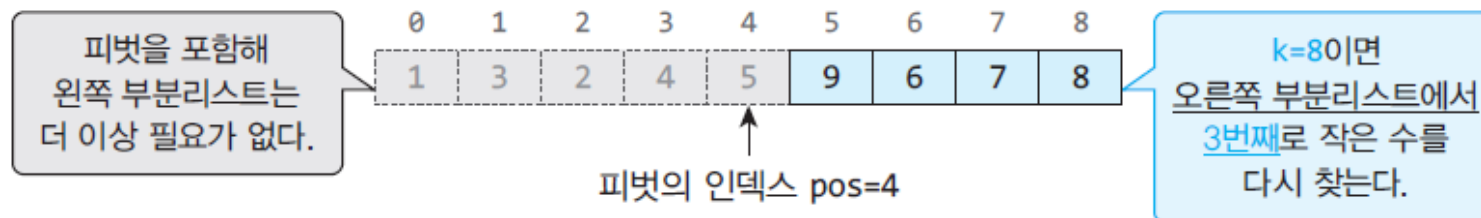
세 가지 경우



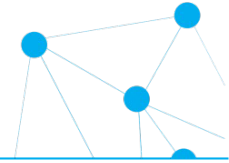
- Case 1) $k = pos+1$: 답은 구해짐
- Case 2) $k < pos+1$: 답은 왼쪽 부분 리스트에



- Case3: $k > pos+1$: 답은 오른쪽 부분 리스트에



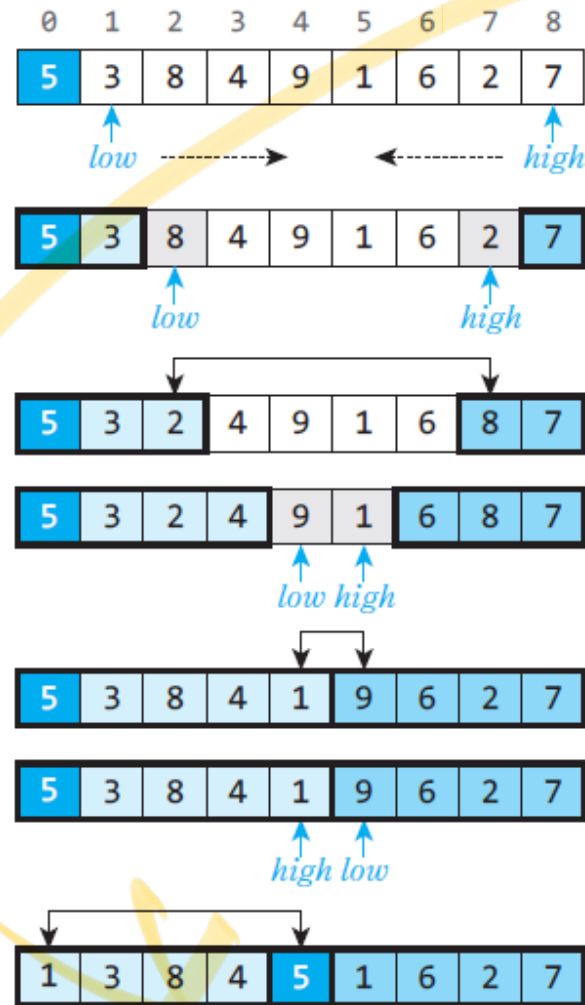
quick_select() 알고리즘



```
qsel(A, left, right, k)
    pos = partition(A, left, right)
    if (pos+1 == left+k)
        return A[pos]
    elif (pos+1 > left+k)
        return qsel(A, left, pos-1, k)
    else: return qsel(A, pos+1, right, k-(pos+1-left))
```



호어(Hoare) 분할



5를 피벗으로 선택

$low \leftarrow left+1$

$high \leftarrow right$

low를 피벗보다 큰 항목까지 이동

high를 피벗보다 작은 항목까지 이동

low와 high의 항목 교체

다시 진행

low를 피벗보다 큰 항목까지 이동

high를 피벗보다 작은 항목까지 이동

low와 high의 항목 교체

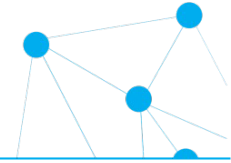
다시 진행

low와 high가 역전됨 → 종료

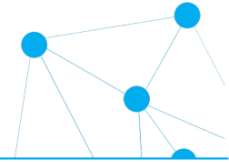
피벗과 high 위치의 항목 교환

[그림 4.18] 피벗을 중심으로 두 개의 리스트로 나누는 호어 분할 과정

partition(): 알고리즘

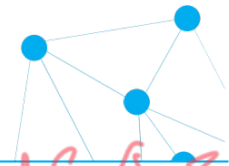


복잡도 분석



- 입력 구성에 따른 차이가 있음
 - 최선
 - 한번의 분할로 답을 찾는 경우
 - $T_{best}(n) \in O(n)$
 - 최악
 - 불균등 분할이 계속 되는 경우
 - $T_{worst}(n) \in O(n^2)$
 - 평균
 - 가정이 필요
 - $T_{avg}(n) \in O(n)$
 - 증명: 172~173쪽 심화학습
- quick_select()
 - 가변적인 크기(variable size decrease)로 문제의 크기가 줄어드는 전형적인 축소 정복 알고리즘

4.6 축소 정복 기법의 추가적인 예



• 위조 동전 찾기: 고정 비율 축소

동일하게 생긴 n 개의 동전과 양팔 저울이 주어졌다. 동전들 중에서 하나는 가짜 동전이고 진짜 동전보다 약간 가볍다. 당연히 양팔 저울은 동전의 무게를 직접 측정할 수 없다. 양쪽의 무게가 같은지 또는 어느 쪽이 더 무거운지 만을 알 수 있을 뿐이다. 이 저울을 최소한의 횟수만 사용하여 어느 동전이 가짜 동전인지를 찾아라.

- 고정 비율 축소(1/2)

$$T(n) = T(\lfloor n/2 \rfloor) + 1 \quad \text{for } n > 1$$

$$T(n) = \lfloor \log_2 n \rfloor \in O(\log_2 n)$$

- 고정 비율 축소(1/3)

$$T(n) = T(\lfloor n/3 \rfloor) + 1 \quad \text{for } n > 1$$

$$O(k) = O(\log_3 n)$$

동전 3개: 1번

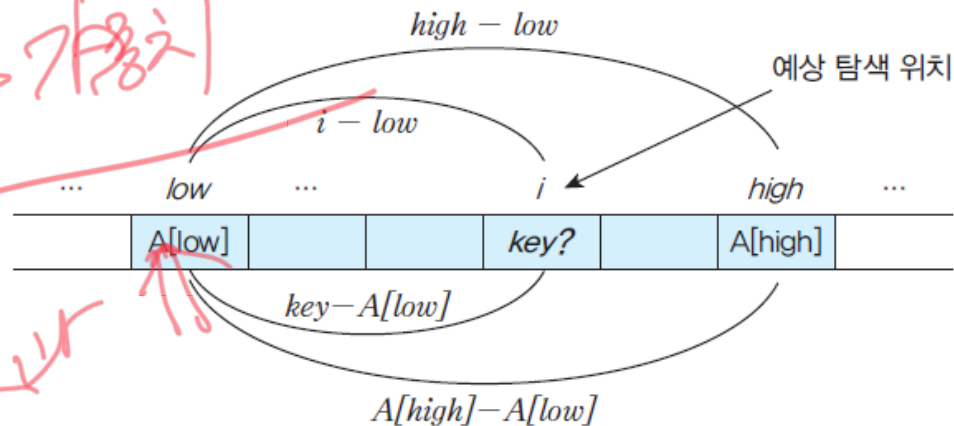
4 ~ 9개: 2번

10 ~ 27개: 3번

28 ~ 81개: 4번

보간 탐색: 가변 크기 축소

- 사전에서 단어를 찾을 때와 같이 탐색키가 존재할 위치를 예측하여 탐색하는 방법

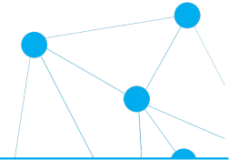


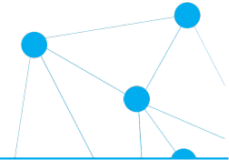
[그림 4.19] 보간 탐색은 찾는 값과 위치가 비례한다고 가정한다.

$$\text{탐색 위치} = low + (high - low) \cdot \frac{key - A[low]}{A[high] - A[low]}$$

```
mid = int(low + (high-low) * (key-A[low].key) / (A[high].key-A[low].key))
```

실습 과제





감사합니다!